Modulo_2

2026-04-01

##Ejercicio 1 de algoritmos

Consigna: Las ultimas 3 cifras de mi DNI son 506, consigna crear una variable que tenga ese numero

```
DNI=506
```

Consigna: Crear un vector que tenga todos los numeros enteros del 1 al 506

#1.4 GENERAR UN VECTOR SECUENCIA

```
secuencia_DNI<-(1:506)
```

##Ejercicio 2

Calcular la suma de todos los valores del vector secuencia_DNI

```
total<-0
valorfinal<-length(secuencia_DNI)
for (i in 1:valorfinal) {
total<-total+i
total
}
total
```

```
## [1] 128271
```

1.5 SERIE DE FIBONACHI

```
# Definimos la cantidad de elementos basados en tu variable DNI
n<-506
fibonacci <- numeric(n)

# Inicializamos los dos primeros valores
fibonacci[1] <- 0
fibonacci[2] <- 1

# Generamos la serie mediante un bucle for
for (i in 3:n) {
  fibonacci[i] <- fibonacci[i-1] + fibonacci[i-2]
}

# Mostramos el resultado
print(fibonacci)
```

```
##   [1]  0.000000e+00  1.000000e+00  1.000000e+00  2.000000e+00  3.000000e+00
##   [6]  5.000000e+00  8.000000e+00  1.300000e+01  2.100000e+01  3.400000e+01
##  [11]  5.500000e+01  8.900000e+01  1.440000e+02  2.330000e+02  3.770000e+02
##  [16]  6.100000e+02  9.870000e+02  1.597000e+03  2.584000e+03  4.181000e+03
##  [21]  6.765000e+03  1.094600e+04  1.771100e+04  2.865700e+04  4.636800e+04
```

```
##  [26]  7.502500e+04  1.213930e+05  1.964180e+05  3.178110e+05  5.142290e+05
##  [31]  8.320400e+05  1.346269e+06  2.178309e+06  3.524578e+06  5.702887e+06
##  [36]  9.227465e+06  1.493035e+07  2.415782e+07  3.908817e+07  6.324599e+07
##  [41]  1.023342e+08  1.655801e+08  2.679143e+08  4.334944e+08  7.014087e+08
##  [46]  1.134903e+09  1.836312e+09  2.971215e+09  4.807527e+09  7.778742e+09
##  [51]  1.258627e+10  2.036501e+10  3.295128e+10  5.331629e+10  8.626757e+10
##  [56]  1.395839e+11  2.258514e+11  3.654353e+11  5.912867e+11  9.567220e+11
##  [61]  1.548009e+12  2.504731e+12  4.052740e+12  6.557470e+12  1.061021e+13
##  [66]  1.716768e+13  2.777789e+13  4.494557e+13  7.272346e+13  1.176690e+14
##  [71]  1.903925e+14  3.080615e+14  4.984540e+14  8.065155e+14  1.304970e+15
##  [76]  2.111485e+15  3.416455e+15  5.527940e+15  8.944394e+15  1.447233e+16
##  [81]  2.341673e+16  3.788906e+16  6.130579e+16  9.919485e+16  1.605006e+17
##  [86]  2.596955e+17  4.201961e+17  6.798916e+17  1.100088e+18  1.779979e+18
##  [91]  2.880067e+18  4.660047e+18  7.540114e+18  1.220016e+19  1.974027e+19
##  [96]  3.194043e+19  5.168071e+19  8.362114e+19  1.353019e+20  2.189230e+20
## [101]  3.542248e+20  5.731478e+20  9.273727e+20  1.500521e+21  2.427893e+21
## [106]  3.928414e+21  6.356307e+21  1.028472e+22  1.664103e+22  2.692575e+22
## [111]  4.356678e+22  7.049252e+22  1.140593e+23  1.845518e+23  2.986111e+23
## [116]  4.831630e+23  7.817741e+23  1.264937e+24  2.046711e+24  3.311648e+24
## [121]  5.358359e+24  8.670007e+24  1.402837e+25  2.269837e+25  3.672674e+25
## [126]  5.942511e+25  9.615186e+25  1.555770e+26  2.517288e+26  4.073058e+26
## [131]  6.590346e+26  1.066340e+27  1.725375e+27  2.791715e+27  4.517090e+27
## [136]  7.308806e+27  1.182590e+28  1.913470e+28  3.096060e+28  5.009530e+28
## [141]  8.105590e+28  1.311512e+29  2.122071e+29  3.433583e+29  5.555654e+29
## [146]  8.989237e+29  1.454489e+30  2.353413e+30  3.807902e+30  6.161315e+30
## [151]  9.969217e+30  1.613053e+31  2.609975e+31  4.223028e+31  6.833003e+31
## [156]  1.105603e+32  1.788903e+32  2.894506e+32  4.683410e+32  7.577916e+32
## [161]  1.226133e+33  1.983924e+33  3.210057e+33  5.193981e+33  8.404038e+33
## [166]  1.359802e+34  2.200206e+34  3.560008e+34  5.760213e+34  9.320221e+34
## [171]  1.508043e+35  2.440065e+35  3.948109e+35  6.388174e+35  1.033628e+36
## [176]  1.672446e+36  2.706074e+36  4.378520e+36  7.084594e+36  1.146311e+37
## [181]  1.854771e+37  3.001082e+37  4.855853e+37  7.856935e+37  1.271279e+38
## [186]  2.056972e+38  3.328251e+38  5.385223e+38  8.713475e+38  1.409870e+39
## [191]  2.281217e+39  3.691087e+39  5.972304e+39  9.663391e+39  1.563570e+40
## [196]  2.529909e+40  4.093478e+40  6.623387e+40  1.071687e+41  1.734025e+41
## [201]  2.805712e+41  4.539737e+41  7.345449e+41  1.188519e+42  1.923063e+42
## [206]  3.111582e+42  5.034645e+42  8.146227e+42  1.318087e+43  2.132710e+43
## [211]  3.450797e+43  5.583507e+43  9.034305e+43  1.461781e+44  2.365212e+44
## [216]  3.826993e+44  6.192205e+44  1.001920e+45  1.621140e+45  2.623060e+45
## [221]  4.244200e+45  6.867260e+45  1.111146e+46  1.797872e+46  2.909018e+46
## [226]  4.706890e+46  7.615908e+46  1.232280e+47  1.993871e+47  3.226150e+47
## [231]  5.220021e+47  8.446172e+47  1.366619e+48  2.211236e+48  3.577856e+48
## [236]  5.789092e+48  9.366948e+48  1.515604e+49  2.452299e+49  3.967903e+49
## [241]  6.420201e+49  1.038810e+50  1.680831e+50  2.719641e+50  4.400472e+50
## [246]  7.120113e+50  1.152058e+51  1.864070e+51  3.016128e+51  4.880198e+51
## [251]  7.896326e+51  1.277652e+52  2.067285e+52  3.344937e+52  5.412222e+52
## [256]  8.757160e+52  1.416938e+53  2.292654e+53  3.709592e+53  6.002246e+53
## [261]  9.711839e+53  1.571409e+54  2.542592e+54  4.114001e+54  6.656593e+54
## [266]  1.077059e+55  1.742719e+55  2.819778e+55  4.562497e+55  7.382275e+55
## [271]  1.194477e+56  1.932705e+56  3.127182e+56  5.059887e+56  8.187069e+56
## [276]  1.324696e+57  2.143402e+57  3.468098e+57  5.611500e+57  9.079598e+57
## [281]  1.469110e+58  2.377070e+58  3.846179e+58  6.223249e+58  1.006943e+59
## [286]  1.629268e+59  2.636211e+59  4.265478e+59  6.901689e+59  1.116717e+60
## [291]  1.806886e+60  2.923602e+60  4.730488e+60  7.654090e+60  1.238458e+61
```

```
## [296]  2.003867e+61  3.242325e+61  5.246192e+61  8.488516e+61  1.373471e+62
## [301]  2.222322e+62  3.595793e+62  5.818116e+62  9.413909e+62  1.523202e+63
## [306]  2.464593e+63  3.987796e+63  6.452389e+63  1.044019e+64  1.689257e+64
## [311]  2.733276e+64  4.422533e+64  7.155809e+64  1.157834e+65  1.873415e+65
## [316]  3.031249e+65  4.904665e+65  7.935914e+65  1.284058e+66  2.077649e+66
## [321]  3.361707e+66  5.439356e+66  8.801064e+66  1.424042e+67  2.304148e+67
## [326]  3.728190e+67  6.032339e+67  9.760529e+67  1.579287e+68  2.555340e+68
## [331]  4.134626e+68  6.689966e+68  1.082459e+69  1.751456e+69  2.833915e+69
## [336]  4.585371e+69  7.419286e+69  1.200466e+70  1.942394e+70  3.142860e+70
## [341]  5.085254e+70  8.228114e+70  1.331337e+71  2.154148e+71  3.485485e+71
## [346]  5.639634e+71  9.125119e+71  1.476475e+72  2.388987e+72  3.865462e+72
## [351]  6.254449e+72  1.011991e+73  1.637436e+73  2.649427e+73  4.286863e+73
## [356]  6.936291e+73  1.122315e+74  1.815944e+74  2.938260e+74  4.754204e+74
## [361]  7.692464e+74  1.244667e+75  2.013913e+75  3.258580e+75  5.272493e+75
## [366]  8.531074e+75  1.380357e+76  2.233464e+76  3.613821e+76  5.847285e+76
## [371]  9.461106e+76  1.530839e+77  2.476950e+77  4.007789e+77  6.484738e+77
## [376]  1.049253e+78  1.697727e+78  2.746979e+78  4.444706e+78  7.191685e+78
## [381]  1.163639e+79  1.882808e+79  3.046447e+79  4.929254e+79  7.975701e+79
## [386]  1.290495e+80  2.088066e+80  3.378561e+80  5.466627e+80  8.845188e+80
## [391]  1.431181e+81  2.315700e+81  3.746882e+81  6.062582e+81  9.809464e+81
## [396]  1.587205e+82  2.568151e+82  4.155355e+82  6.723506e+82  1.087886e+83
## [401]  1.760237e+83  2.848123e+83  4.608360e+83  7.456483e+83  1.206484e+84
## [406]  1.952133e+84  3.158617e+84  5.110749e+84  8.269366e+84  1.338012e+85
## [411]  2.164948e+85  3.502960e+85  5.667908e+85  9.170868e+85  1.483878e+86
## [416]  2.400964e+86  3.884842e+86  6.285806e+86  1.017065e+87  1.645645e+87
## [421]  2.662710e+87  4.308356e+87  6.971066e+87  1.127942e+88  1.825049e+88
## [426]  2.952991e+88  4.778040e+88  7.731030e+88  1.250907e+89  2.024010e+89
## [431]  3.274917e+89  5.298927e+89  8.573844e+89  1.387277e+90  2.244662e+90
## [436]  3.631939e+90  5.876600e+90  9.508539e+90  1.538514e+91  2.489368e+91
## [441]  4.027882e+91  6.517250e+91  1.054513e+92  1.706238e+92  2.760751e+92
## [446]  4.466989e+92  7.227740e+92  1.169473e+93  1.892247e+93  3.061720e+93
## [451]  4.953967e+93  8.015687e+93  1.296965e+94  2.098534e+94  3.395500e+94
## [456]  5.494034e+94  8.889533e+94  1.438357e+95  2.327310e+95  3.765667e+95
## [461]  6.092977e+95  9.858643e+95  1.595162e+96  2.581026e+96  4.176188e+96
## [466]  6.757215e+96  1.093340e+97  1.769062e+97  2.862402e+97  4.631464e+97
## [471]  7.493866e+97  1.212533e+98  1.961920e+98  3.174453e+98  5.136372e+98
## [476]  8.310825e+98  1.344720e+99  2.175802e+99  3.520522e+99  5.696324e+99
## [481]  9.216846e+99 1.491317e+100 2.413002e+100 3.904318e+100 6.317320e+100
## [486] 1.022164e+101 1.653896e+101 2.676060e+101 4.329956e+101 7.006015e+101
## [491] 1.133597e+102 1.834199e+102 2.967796e+102 4.801994e+102 7.769790e+102
## [496] 1.257178e+103 2.034157e+103 3.291336e+103 5.325493e+103 8.616829e+103
## [501] 1.394232e+104 2.255915e+104 3.650147e+104 5.906063e+104 9.556210e+104
## [506] 1.546227e+105
```

1.6 DEFINICION MATEMATICA RECURRENTE

```r
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {
```

```
  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
```

```
## [1] 1346269
```

```
i
```

```
## [1] 30
```

1.7 Comparacion del método burbuja y el método sort de R

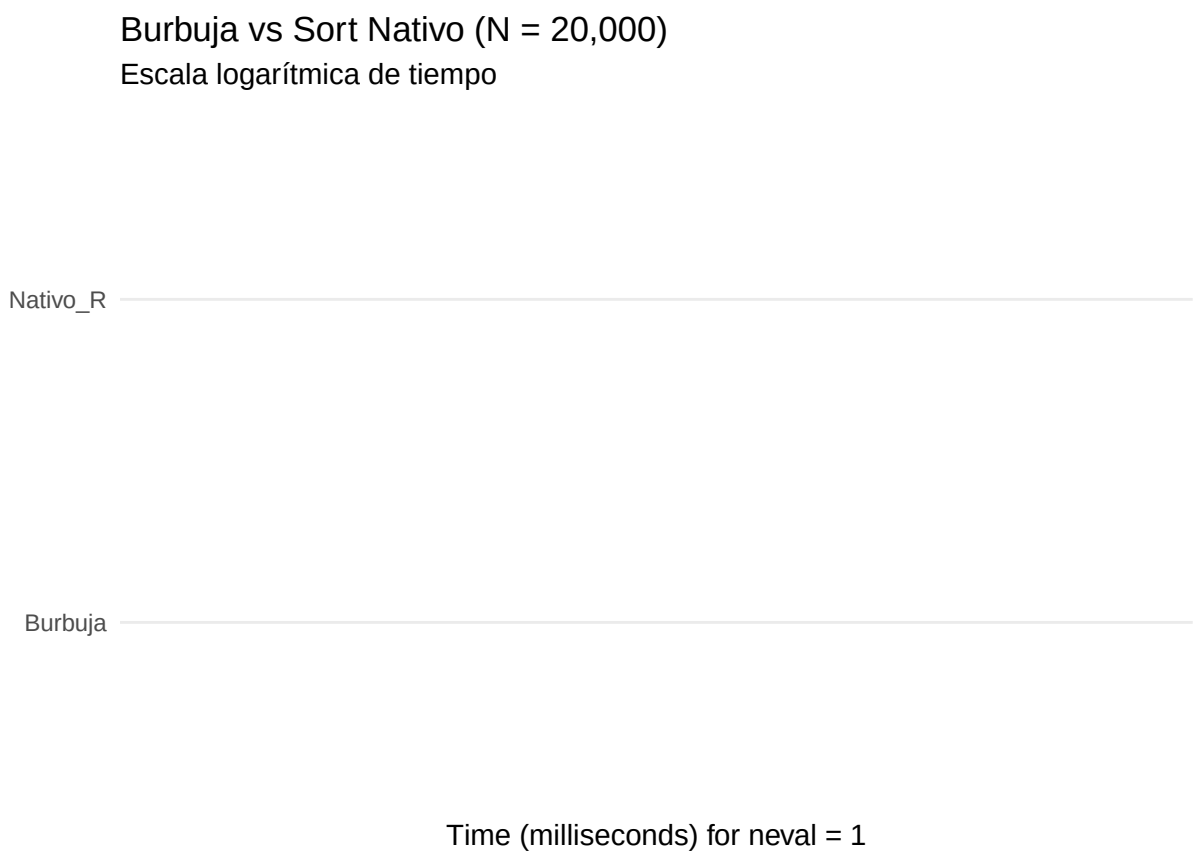
```
library(microbenchmark)
library(ggplot2)

# 1. Definimos la función de Burbuja
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n - 1)) {
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# 2. Creamos la muestra de 20,000 números aleatorios
set.seed(506) # Usando tu DNI como semilla
muestra_grande <- runif(20000)

# 3. Ejecutamos el microbenchmark
# Nota: Reducimos 'times' a 1 porque Burbuja es muy lento con 20k elementos
comparativa_20k <- microbenchmark(
  Burbuja = bubble_sort(muestra_grande),
  Nativo_R = sort(muestra_grande),
  times = 1
)

# 4. Ver los resultados en consola
print(comparativa_20k)
```

```
## Unit: milliseconds
##      expr          min           lq         mean       median           uq
##   Burbuja 21409.675308 21409.675308 21409.675308 21409.675308 21409.675308
##  Nativo_R     1.037451     1.037451     1.037451     1.037451     1.037451
##           max neval
##  21409.675308     1
```

```
##      1.037451     1
```

```r
# 5. Gráfico de Violín para visualizar la diferencia
autoplot(comparativa_20k) +
  labs(title = "Burbuja vs Sort Nativo (N = 20,000)",
       subtitle = "Escala logarítmica de tiempo",
       y = "Tiempo de ejecución",
       x = "Algoritmo") +
  theme_minimal()
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
##   Please report the issue at
##   <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

## Warning: Groups with fewer than two datapoints have been dropped.
## i Set `drop = FALSE` to consider such groups for position adjustment purposes.
## Groups with fewer than two datapoints have been dropped.
## i Set `drop = FALSE` to consider such groups for position adjustment purposes.

## Warning in max(data$density, na.rm = TRUE): no non-missing arguments to max;
## returning -Inf

## Warning: Computation failed in `stat_ydensity()`.
## Caused by error in `$<-.data.frame`:
## ! replacement has 1 row, data has 0
```



##Ejercicio 3

Repetir el ejercicio anterior pero en Python

```
suma = 0

for i in range(1, 507):
    suma+=i
print(suma)
```

```
## 128271
```

##Ejercicio 4 ¿Cuanto tarda en correr el ejercicio 2?

```
inicio<-Sys.time()

total<-0
valorfinal<-10000000*length(secuencia_DNI)
for (i in 1:valorfinal) {
total<-total+i
total
}
total
```

```
## [1] 1.28018e+19
```

```
final<-Sys.time()
final-inicio
```

```
## Time difference of 1.877184 mins
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```
secuencia_DNI<-(1:506)
valorfinal <- 1e8 * length(secuencia_DNI)

inicio1 <- Sys.time()

total1 <- 0
for (i in 1:valorfinal) {
  total1 <- total1 + i
}

final1 <- Sys.time()
tiempo1 <- final1 - inicio1

cat("Resultado 1:", total1, "\n")
```

```
## Resultado 1: 1.28018e+21
```

```
cat("Tiempo Fuerza Bruta:", tiempo1, "segundos\n")
```

```
## Tiempo Fuerza Bruta: 14.31104 segundos
```

Ejercicio 6

1.7.2 Ordenamiento burbuja

```
# --- Script de Muestreo y Ordenamiento en R ---

# 1. Definir la muestra de 100 números (del 1 al 100)
poblacion <- 1:100

# 2. Tomar 10 números aleatorios de esa muestra
# set.seed() es opcional, sirve para que los números no cambien cada vez que corres el código
# set.seed(123)

muestra_aleatoria <- sample(poblacion, 10)

# 3. Ordenar los números de menor a mayor
muestra_ordenada <- sort(muestra_aleatoria)

# --- MOSTRAR RESULTADOS EN CONSOLA ---

cat("1. Muestra de 10 números aleatorios (sin orden):\n")
```

```
## 1. Muestra de 10 números aleatorios (sin orden):
```

```
print(muestra_aleatoria)
```

```
##  [1]   3  16  50  17  32 100  37  31  63  77
```

```
cat("\n2. Muestra ordenada de menor a mayor:\n")
```

```
##
## 2. Muestra ordenada de menor a mayor:
```

```r
print(muestra_ordenada)
```

```
##  [1]   3  16  17  31  32  37  50  63  77 100
```

1.8 La penitencia de Newton

```r
library(microbenchmark)

# Desactivar notación científica
options(scipen = 999)

n <- 1010

# Algoritmo A: Fuerza Bruta (El "castigo" del profesor)
suma_bucle <- function(n) {
  resultado <- 0
  for (i in 1:n) {
    resultado <- resultado + i
  }
  return(resultado)
  resultado
}

# Algoritmo B: Fórmula Matemática (La "astucia" de Newton/Gauss)
suma_formula <- function(n) {
  return((n * (n + 1)) / 2)
  suma_formula
}

# Verificación de eficiencia
comparativa_newton <- microbenchmark(
  Fuerza_Bruta = suma_bucle(n),
  Formula_Gauss = suma_formula(n),
  times = 10
)

print(comparativa_newton)
```

```
## Unit: nanoseconds
##           expr   min    lq     mean median    uq     max neval
##   Fuerza_Bruta 16740 16750 260800.2  16875 17790 2453802    10
##  Formula_Gauss   300   300 193513.1    310   400 1916711    10
```

2 Agrupacion K-mean

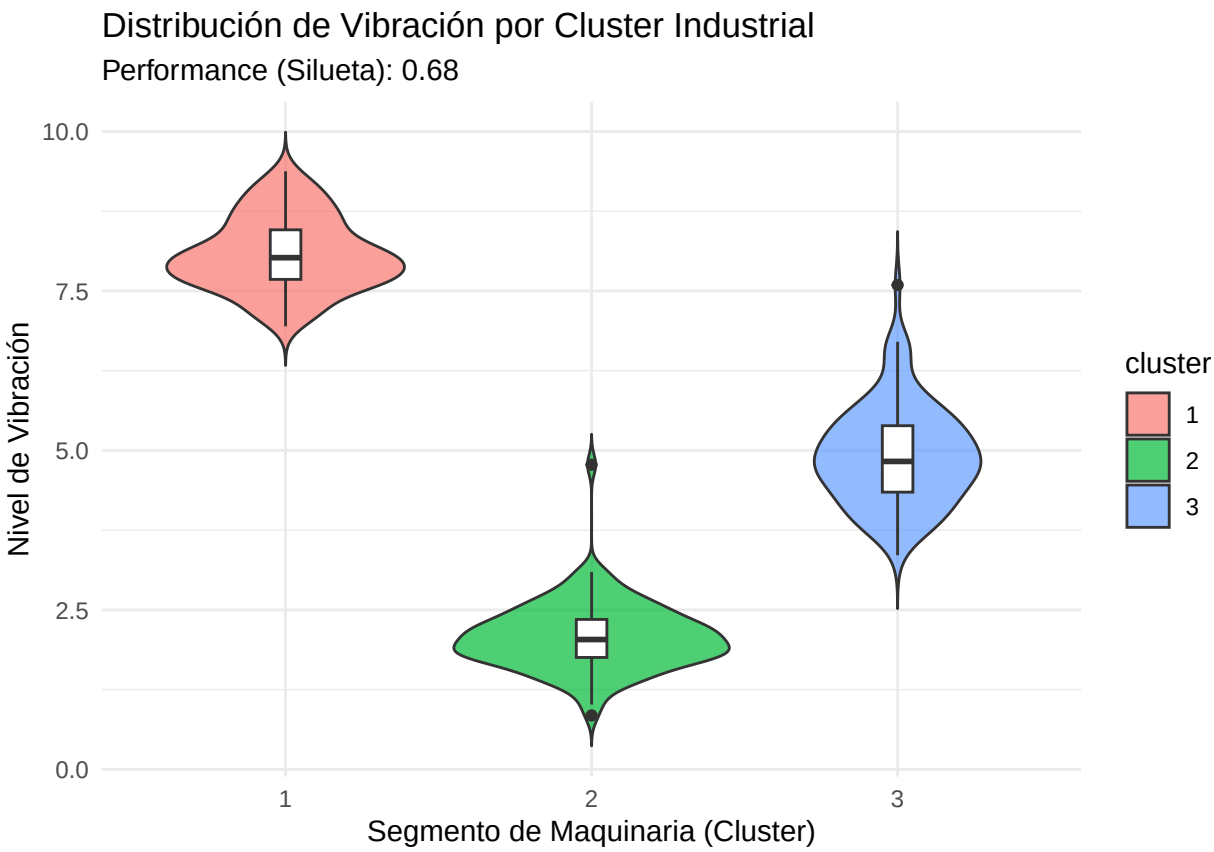
Medición de Performance: Agrupamiento K-MeansEn el análisis de datos industriales, no basta con agrupar los componentes; es imperativo evaluar qué tan "buenas" son esas agrupaciones. A diferencia del aprendizaje supervisado, donde comparamos predicciones contra etiquetas reales, en el Clustering medimos la calidad mediante la cohesión interna y la separación externa.1. Métricas de Evaluación InternaPara determinar si la segmentación de maquinaria tiene sentido estadístico, se utilizan principalmente dos indicadores:WSS (Within-Cluster Sum of Squares): Mide la suma de los cuadrados dentro de cada grupo. Representa la variabilidad interna. Un WSS bajo indica que los componentes dentro de un cluster son muy similares entre sí (alta cohesión).Coeficiente de Silueta (s_i): Es una medida que combina cohesión y separación. Su valor oscila entre -1 y 1:1: El componente está muy bien ubicado en su cluster.0: El componente está en el límite entre

dos clusters.−1: El componente probablemente ha sido asignado al grupo incorrecto.2. El Gráfico de Violín como Herramienta de ValidaciónSi bien métricas como el "Método del Codo" nos ayudan a elegir el número óptimo de clusters (k), el gráfico de violín es esencial para la interpretación física de los resultados.Este gráfico combina un diagrama de caja (boxplot) con una curva de densidad de probabilidad. En el contexto de mantenimiento industrial, nos permite visualizar:Distribución y Densidad: Podemos ver dónde se concentra la mayoría de los datos de sensores (vibración, temperatura) para cada grupo.Separación de Grupos: Si los "cuerpos" de los violines de diferentes clusters no se solapan significativamente, el algoritmo ha logrado identificar estados operativos claramente distintos (ej. Estado Normal vs. Falla Inminente).Detección de Anomalías: La extensión de las "colas" del violín permite identificar máquinas que, aunque pertenecen a un grupo, presentan valores extremos que requieren atención.

```r
# Librerías necesarias
library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)
```

```
## El coeficiente de silueta promedio es: 0.6808356
```

```r
# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()
```



R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

R es un lenguaje muy parecido a matlab que en general pueden ser matrices

```
a=38
a=40
b=60
c=b-a
c
```

```
## [1] 20
```

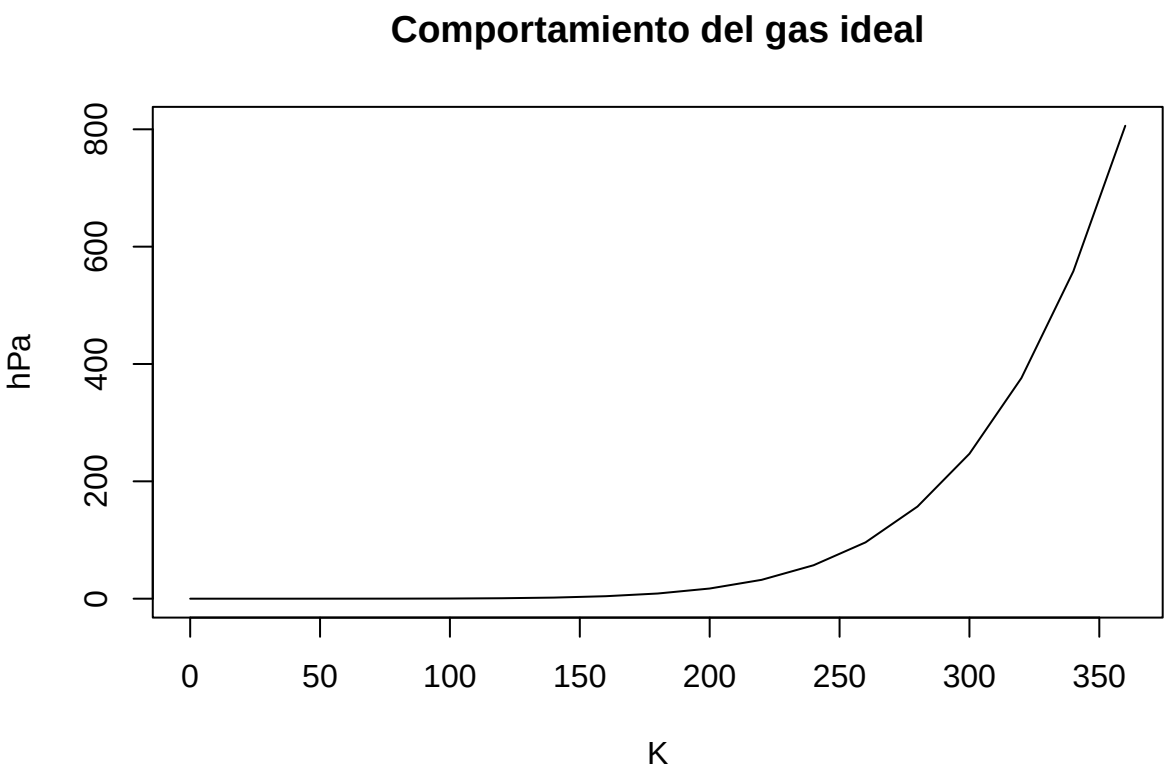
```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

Including Plots

You can also embed plots, for example:

```
plot(pressure, type="l", main='Comportamiento del gas ideal',ylab='hPa',xlab='K')
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```
Y=40
alpha<-30
```

##uso de datasets o base de datos internos de R

Usando el comando de data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan en tablas de las cuales podemos elegir columnas especificas si escribimos $ despues del comando de informacion especifico y elegimos la columna de datos deseada

Comando plot

Es un comando que permite graficar caulquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir *?plot* en la consola, lo cual abriria una ventana que explicara en detalle el comando.

Funciones estadisticas

%explicar despues

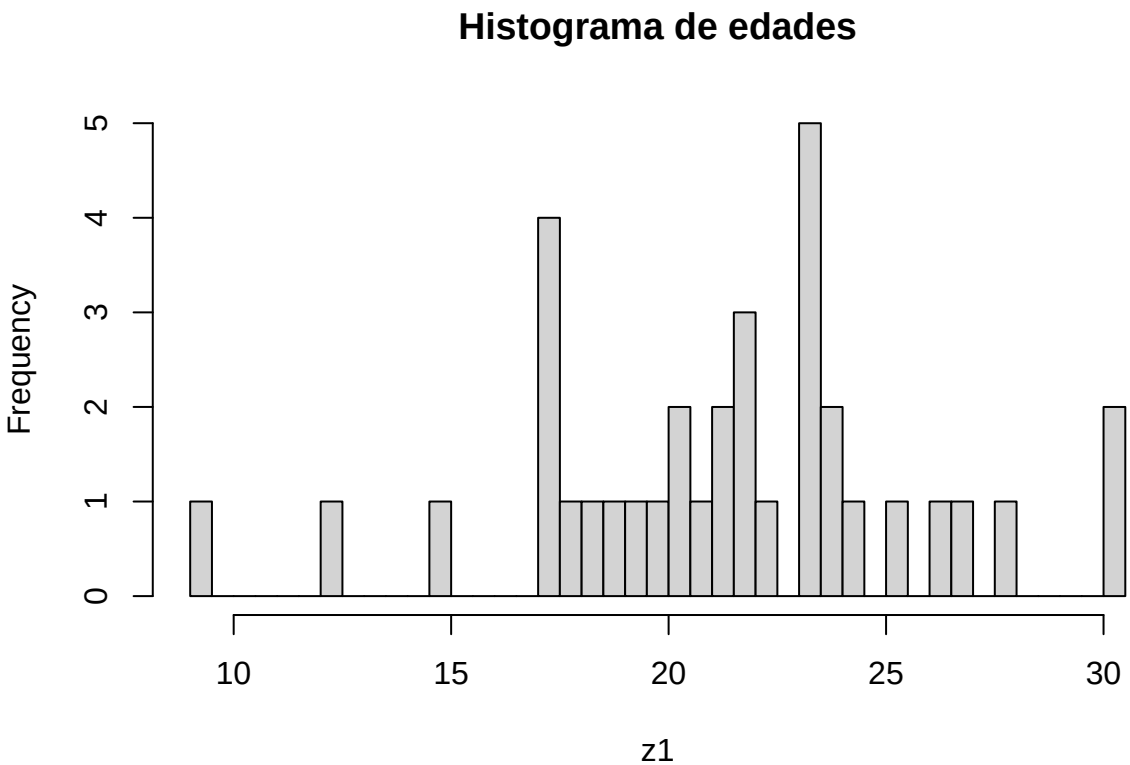
```
z1<-rnorm(35,22,5)
z1
```

```
##  [1] 20.168980 20.529994 21.884387 23.811997 19.666428 23.492568 17.236459
```

```
##  [8] 25.337703 22.436193 21.453953 18.612539 30.494113 23.929155 23.233785
## [15]  9.065341 12.184485 21.693476 26.476263 21.859706 17.001265 21.201980
## [22] 19.361532 23.080782 17.405225 17.497027 23.443044 26.644424 30.365236
## [29] 23.427572 20.137088 14.830586 17.577660 27.620915 18.385971 24.313325
```

%350 valores, con un promedio alrededor de 22 con una desviacion estandar de 5

```r
hist(z1,main='Histograma de edades',breaks=50)
```

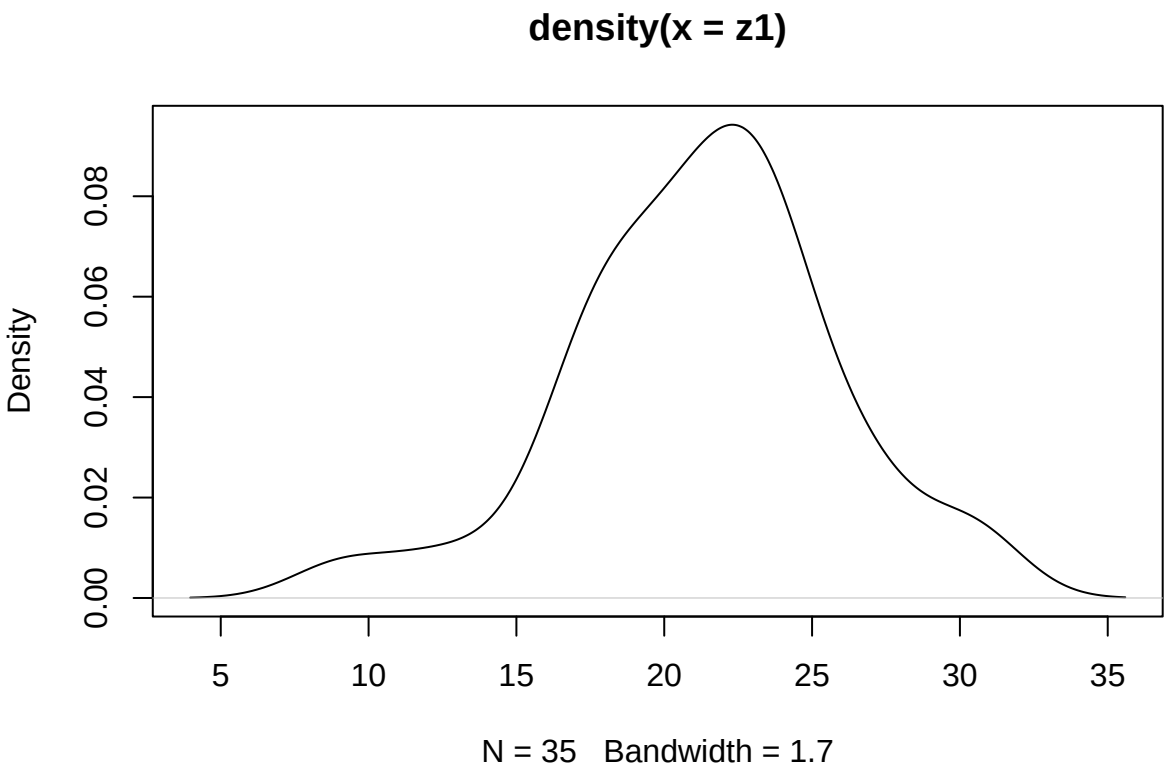



```r
density(z1,type="d")
```

```
## Warning: In density.default(z1, type = "d") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "d")
##
## Data: z1 (35 obs.);  Bandwidth 'bw' = 1.7
##
##        x                y
##  Min.   : 3.966   Min.   :0.00007487
##  1st Qu.:11.873   1st Qu.:0.00719131
##  Median :19.780   Median :0.01714470
##  Mean   :19.780   Mean   :0.03155329
##  3rd Qu.:27.687   3rd Qu.:0.05808773
##  Max.   :35.593   Max.   :0.09424435
```

```r
plot(density(z1))
```




```r
## Ejercicio: Operaciones con Matrices (Matriz Inversa)

# 1. Definimos los datos basados en tu DNI (506)
# Usaremos las cifras 5, 0, 6 y combinaciones para completar la matriz 3x3
dni_cifras <- c(5, 0, 6, 6, 5, 0, 1, 2, 3) # Agregamos valores extra para completar 9 elementos

# 2. Creamos la matriz 3x3
# byrow = TRUE organiza los números por filas
mi_matriz <- matrix(dni_cifras, nrow = 3, ncol = 3, byrow = TRUE)

cat("--- Matriz Original (basada en DNI) ---\n")
```

```
## --- Matriz Original (basada en DNI) ---
```

```r
print(mi_matriz)
```

```
##      [,1] [,2] [,3]
## [1,]    5    0    6
## [2,]    6    5    0
## [3,]    1    2    3
```

```r
# 3. Verificamos el determinante
# Una matriz solo tiene inversa si su determinante es distinto de cero
det_matriz <- det(mi_matriz)

cat("\nDeterminante de la matriz:", det_matriz, "\n")
```

```
##
## Determinante de la matriz: 117
```

```r
# 4. Cálculo de la inversa
# Usamos una estructura condicional (if) para verificar la viabilidad
if (det_matriz != 0) {
  matriz_inversa <- solve(mi_matriz)

  cat("\n--- Matriz Inversa ---\n")
  print(matriz_inversa)

  # 5. Verificación (Matriz * Inversa = Identidad)
  cat("\n--- Verificación (Producto debe ser la Identidad) ---\n")
  print(round(mi_matriz %*% matriz_inversa, 2))

} else {
  cat("\nLa matriz no es invertible (el determinante es 0).\n")
}
```

```
##
## --- Matriz Inversa ---
##             [,1]        [,2]       [,3]
## [1,]  0.12820513  0.10256410 -0.2564103
## [2,] -0.15384615  0.07692308  0.3076923
## [3,]  0.05982906 -0.08547009  0.2136752
##
## --- Verificación (Producto debe ser la Identidad) ---
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1
```